

Infix to Postfix Conversion & Recursion

Q. Discuss the application of Stacks in evaluating mathematical expressions. Explain the process of Infix to Postfix conversion, Postfix evaluation, and how stacks handle Recursion.


Definition to Remember

Infix notation (`A + B`) is human-readable but hard for computers. **Postfix notation** (`A B +`) places operators after operands — no parentheses or precedence rules needed. A **Stack** is used to convert Infix → Postfix and to evaluate Postfix expressions. Stacks also manage **Recursion** through the **Call Stack**.

1. Operator Precedence (for conversion)

Operator	Precedence
<code>^</code> (Exponentiation)	Highest (3)
<code>*</code> , <code>/</code>	Medium (2)
<code>+</code> , <code>-</code>	Lowest (1)

2. Infix to Postfix Conversion Algorithm

1. Scan expression left to right.
2. **Operand** → immediately add to **Output**.
3. **Operator** → push to stack; pop operators with $\geq$ precedence to output first.
4. `(` → push to stack; `)` → pop to output until `(` is found (don't output `(`).
5. End of expression → pop all remaining operators to output.

Example: `A * (B + C)`

Symbol	Stack	Output
<code>A</code>	—	<code>A</code>
<code>*</code>	<code>*</code>	<code>A</code>
<code>(</code>	<code>* (</code>	<code>A</code>
<code>B</code>	<code>* (</code>	<code>A B</code>
<code>+</code>	<code>* (+</code>	<code>A B</code>
<code>C</code>	<code>* (+</code>	<code>A B C</code>
<code>)</code>	<code>*</code>	<code>A B C +</code>
End	—	<code>A B C + *</code>

3. Postfix Evaluation Algorithm

1. Scan postfix string left to right.
2. **Operand** → push onto stack.
3. **Operator** → pop top two operands, apply operator, push result back.
4. Final result is at the **top of the stack**.

Example: `5 3 + 2 *` → $(5+3)2 = 16^*$

4. Recursion and the Call Stack

- When a function calls itself, the OS uses the **Call Stack** to manage it.
- Each call creates a **Stack Frame** containing: local variables, parameters, return address.
- Frames are **pushed** on each call, **popped** when the function returns.
- **Stack Overflow**: occurs if base case is missing or recursion is too deep.

```
factorial(3)
```

factorial(3) → 3	← top (most recent call)
factorial(2) → 2	
factorial(1) → 1	← base case returns first

★ Must-Write Points (for 10 marks)

1. Infix = human-readable; Postfix = machine-friendly (no parentheses or precedence needed).
2. Infix-to-Postfix: operand→output, operator→stack (with precedence), `(` →push, `)` →pop to output.
3. Precedence order: `^` > `*`, `/` > `+`, `-`.
4. Postfix evaluation: operand→push, operator→pop two, apply, push result.
5. Final result of postfix evaluation is at the **top of the stack**.
6. Recursion uses the **Call Stack** — each call creates a Stack Frame (pushed/popped).
7. **Stack Overflow** in recursion: no base case or recursion too deep.

⚡ Quick Recall

Infix→Postfix (operand→output, operator→stack by precedence, brackets) → **Postfix Eval** (operand→push, operator→pop 2, compute, push) → **Recursion** (Call Stack → Stack Frames → base case pops all)